Technical Report: Agentic LLM Web Application

Author: Margot Zhao
Date: April 2026
Stack: FastAPI · Claude Opus 4.7 · Next.js · Supabase · Render · Vercel

---

Table of Contents

1. Problem Statement
2. System Design
3. Why an Agentic Approach
4. Technology Choices
5. Observability
6. Metrics
7. Evaluation
8. Deployment
9. Reflection

---

1. Problem Statement

Large language models have broad factual knowledge but two structural weaknesses that matter for a general-purpose assistant: their training data has a cutoff date, and they produce unreliable arithmetic when reasoning about numbers symbolically. A user asking "What is the current price of gold? Now square that number" needs both a live web lookup and a deterministic computation — neither of which a static LLM call can provide reliably.

The design goal was to build a conversational assistant that:

- Answers direct knowledge questions without unnecessary tool calls.
- Looks up current information when a question requires it.
- Performs exact arithmetic when a question involves calculation.
- Handles multi-step questions that require both tools in sequence.
- Streams answers incrementally so the interface feels responsive.
- Logs all activity to a persistent store for observability and debugging.

The challenge is that the routing decision — which tool to use, or none at all — cannot be determined by keyword matching or a static decision tree. It requires understanding the intent of the question in context.

---

2. System Design

2.1 High-level architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│  Browser (Next.js / Vercel)                                          │
```

```
│                                                                  │
│  Chat.tsx                                                        │
│  ├─ POST /chat  {message, history}                               │
│  └─ ReadableStream ← SSE events                                  │
└──────────────────────────┬──────────────────────────────────────┘
                           │ text/event-stream
┌──────────────────────────▼──────────────────────────────────────┐
│  FastAPI backend (Render)                                        │
│                                                                  │
│  main.py                                                         │
│  ├─ generates request_id UUID                                    │
│  └─ delegates to agent.run() → StreamingResponse                 │
│                                                                  │
│  agent.py  (agentic loop)                                        │
│  ├─ anthropic.messages.create(model, tools, messages)            │
│  ├─ stream SSE: text / tool_use / server_tool_use / tool_result  │
│  ├─ execute client-side tools (calculator)                       │
│  └─ loop until stop_reason == "end_turn"                         │
│                                                                  │
│  Anthropic API                                                   │
│  ├─ claude-opus-4-7                                              │
│  ├─ web_search_20260209  (server-side — Anthropic executes)      │
│  └─ calculator           (client-side — backend executes)        │
│                                                                  │
│  Supabase (Postgres)                                             │
│  ├─ chat_logs            one row per completed request           │
│  └─ tool_error_logs      one row per failed client-side tool     │
└──────────────────────────────────────────────────────────────────┘
```

2.2 The agentic loop

The loop in `agent.py` follows the standard tool-use protocol:

1. Send the current message list to `messages.create`.
2. Iterate over the response content blocks:
   - `text` — stream the text to the client.
   - `tool_use` — execute the tool, stream the result, append both the assistant turn and the `tool_result` turn to the message list.
   - `server_tool_use` — the model ran a server-side tool automatically; stream an informational event but take no action.
3. Check `stop_reason`:
   - `end_turn` — the model is done; break.
   - `tool_use` — tool results were appended; loop.
   - `pause_turn` — the server-side tool hit its iteration limit; re-send the assistant turn without adding a new user message (the API detects the trailing `server_tool_use` block and resumes).

```
messages = [user_turn]

loop:
  response ← messages.create(messages)
```

```
  for block in response.content:
    if text         → yield SSE text event
    if tool_use     → execute tool; yield tool_use + tool_result SSE; append turns
    if server_tool_use → yield informational SSE

  if stop_reason == end_turn  → break
  if stop_reason == pause_turn → append assistant turn; continue (no new user
turn)
  if stop_reason == tool_use  → loop
```

2.3 Tool taxonomy

Tool	Type	Execution	Use case
`web_search_20260209`	Server-side	Anthropic's infrastructure	Current facts, prices, news
`calculator`	Client-side	Backend `eval()` with math whitelist	Exact arithmetic

The distinction matters for the loop. Server-side tools execute between API calls without round-tripping to the backend; the backend never sees their input or output, only the `server_tool_use` block that records what ran. Client-side tools require the backend to execute the function and return a `tool_result` message before the next API call.

2.4 SSE event protocol

All streaming is encoded as Server-Sent Events over a single HTTP response. The frontend parses a buffer of partial chunks, splitting on `\n` and popping the last (potentially incomplete) line back into the buffer for the next read.

Event type	Meaning
`text`	Incremental assistant text delta
`tool_use`	Client-side tool is about to run (name + input)
`server_tool_use`	Server-side tool ran (name + input, informational)
`tool_result`	Client-side tool finished (result string + `success` boolean)
`error`	Unrecoverable loop error; loop aborted
`done`	Final event; always emitted, even after errors

The `success` boolean on `tool_result` lets the frontend style failed tool calls distinctly without re-parsing the result string.

2.5 Frontend rendering

Each assistant message carries a `toolEvents` array that the UI updates in place as SSE events arrive. Tool calls render as bordered cards above the text response, with three mutually exclusive states:

- **Running** — tool_use received, no tool_result yet; animated pulse indicator.

- **Success** — tool_result received with `success: true`; green checkmark.
- **Failed** — tool_result received with `success: false`; red border and error text.

Server-side tool cards show a "server-side" label and no result (the backend never receives the output). Text content renders below the tool trace with a streaming cursor while the response is in flight.

---

3. Why an Agentic Approach

3.1 Alternatives considered

Static routing (if/else on keywords): A rule-based router could detect words like "search", "calculate", or "price of" and dispatch accordingly. This breaks on paraphrasing, multi-intent questions, and cases where the model should answer directly from training knowledge rather than searching (e.g., asking for the square root of 144 does not need a calculator tool call, and asking "who wrote Hamlet" does not need a web search).

Always-on tools (call both tools every time): Forcing every request through both a web search and a calculator adds latency, cost, and irrelevant tool output that the model must then filter out. It also degrades responses for questions where neither tool adds value.

Separate specialized endpoints: An endpoint for search questions, one for math questions, one for general questions would require the client to pre-classify — pushing the same routing problem upstream.

3.2 Why agentic routing is the right fit

The model has superior understanding of when a tool is actually needed. It distinguishes "what is 2 + 2" (answerable directly) from "what is the current unemployment rate, and how does it compare to the pre-pandemic rate as a percentage change" (requires a search, then a calculation, chained). A static router cannot handle the chaining case at all.

The agentic loop also handles failure gracefully: if a tool returns an error the model sees the `is_error` flag on the `tool_result` and can decide whether to retry with a different input, attempt an alternative approach, or explain the failure in its response. This is not possible when routing is handled outside the model.

3.3 Cost of the agentic approach

The tradeoff is latency and cost. Each tool call adds a full round trip to the Anthropic API. A question that triggers a web search and a calculator call makes three API calls total. The `duration_ms` metric in `chat_logs` captures the total wall-clock time including all tool round trips, which is necessary context when interpreting latency figures.

---

4. Technology Choices

4.1 Model: Claude Opus 4.7

Claude Opus 4.7 is the most capable model in the Claude 4 family at the time of writing. It is the only model that supports the `web_search_20260209` server-side tool, which offloads search infrastructure entirely to Anthropic. Importantly, Opus 4.7 does not accept `temperature`, `top_p`, or `budget_tokens` parameters — the API returns a 400 if these are included. It supports adaptive thinking via `thinking: {type: "adaptive"}`, though this application does not enable it since the tasks do not require extended reasoning chains.

4.2 Backend: FastAPI

FastAPI was chosen for its native `async/await` support, which is necessary for two reasons. First, the agentic loop makes multiple sequential async calls to the Anthropic API; blocking I/O would serialize these across all concurrent users. Second, `StreamingResponse` with an `AsyncGenerator` is the standard pattern for SSE in FastAPI and requires no additional libraries.

The Supabase Python client is synchronous. Rather than switching to an async client or adding `httpx` as a dependency, all Supabase calls are wrapped in `asyncio.to_thread()`, which runs them in a thread pool without blocking the event loop. This is a pragmatic pattern for integrating sync clients in async codebases.

4.3 Calculator: safe eval

The calculator tool evaluates expressions using Python's `eval()` with `__builtins__` set to an empty dict and a whitelist namespace containing only `math` module functions plus `abs`, `round`, `int`, and `float`. This prevents code execution while supporting a useful range of mathematical operations. The function raises `ValueError` on failure rather than returning an error string, which lets the caller distinguish a tool execution error from a successful result of zero or empty string.

4.4 Frontend: Next.js 15 + Tailwind CSS 3

Next.js was chosen for the App Router's streaming capabilities and the straightforward Vercel deployment path. The chat interface is a single client component (`"use client"`) — there is no server-side data fetching, so the App Router's RSC features are not used but also add no complexity. Tailwind CSS handles styling without a component library dependency, keeping the bundle small.

The SSE parsing uses the browser's native `ReadableStream` API rather than the `EventSource` API because `EventSource` does not support `POST` requests. The parser accumulates chunks in a string buffer and splits on newlines, which correctly handles partial chunks delivered by the network.

4.5 Database: Supabase

Supabase provides a hosted Postgres instance with a REST API (PostgREST) and a Python client. Row Level Security is enabled on both tables. The backend uses the `service_role` key, which bypasses RLS, because the logging writes happen server-side and do not correspond to an authenticated user session. The `anon` key is intentionally not used in the backend.

The schema uses `JSONB` for `tool_calls` and `input` columns, which allows the full tool call payload to be stored and queried without defining a rigid schema upfront. The `request_id UUID` foreign-key-style relationship between `chat_logs` and `tool_error_logs` enables JOIN queries to correlate failed tool calls with their parent request.

---

5. Observability

5.1 What is logged

`chat_logs` — written once at the end of each request by the `finally` block in `agent.run()`, ensuring it is always written even when the loop raises an exception.

Column	Type	Content
`request_id`	UUID	Correlates with tool_error_logs
`user_message`	TEXT	The user's input
`assistant_response`	TEXT	Full accumulated text response
`tool_calls`	JSONB	Array of `{name, input, result, success}` for every tool invocation
`model`	TEXT	Model ID used
`input_tokens`	INTEGER	Summed across all loop iterations
`output_tokens`	INTEGER	Summed across all loop iterations
`duration_ms`	INTEGER	Total wall-clock time including all tool round trips
`error`	TEXT	Exception message if the loop failed, else null

`tool_error_logs` — written immediately when a client-side tool raises an exception, before the error is returned to the model. This provides a real-time record of failures independent of the end-of-request summary.

Column	Type	Content
`request_id`	UUID	Join key to chat_logs
`tool_name`	TEXT	Which tool failed
`input`	JSONB	The input that caused the failure
`error`	TEXT	Exception message
`duration_ms`	INTEGER	Time spent before the failure

5.2 Logging architecture decisions

Immediate vs batched tool error logging. Tool errors are logged immediately rather than batched with the end-of-request summary. This matters because the model may recover from a tool failure and produce a successful response; if the backend crashes afterward, the tool error would be lost in a batched design. Immediate logging also enables real-time alerting on tool error rates without waiting for the request to complete.

Token counting across loop iterations. The loop accumulates `input_tokens` and `output_tokens` across all API calls in a single request. This is necessary because a multi-tool request makes multiple `messages.create` calls, each of which returns its own usage object. The per-request sum in `chat_logs` is what matters for cost attribution.

Non-blocking logging. All Supabase writes run in a thread pool via `asyncio.to_thread`. A logging failure never surfaces to the user — exceptions in `_log_tool_error` and `_log_request` are caught and printed to stderr only. This is a deliberate choice: observability infrastructure should not degrade the user-facing request.

5.3 Querying for debugging

To find all requests where a tool failed and see the surrounding context:

```sql
SELECT
  c.created_at,
  c.user_message,
  c.assistant_response,
  t.tool_name,
  t.input,
  t.error,
  t.duration_ms AS tool_duration_ms,
  c.duration_ms AS total_duration_ms
FROM tool_error_logs t
JOIN chat_logs c ON c.request_id = t.request_id
ORDER BY t.created_at DESC;
```

---

6. Metrics

The system tracks two primary metrics that together cover both quality and operational behavior, plus supporting context values. Both are live and exposed at the `/metrics` endpoint, which queries `chat_logs` and `feedback` in parallel.

6.1 Quality metric: satisfaction rate

What it is: The fraction of rated responses that received a thumbs-up, computed as `positive_ratings / total_ratings`.

Why it matters: Satisfaction rate is a direct signal of answer usefulness from the user's perspective. It captures failures that purely operational metrics miss — a response may have low latency and invoke the correct tool yet still be unsatisfying because the answer was incomplete, badly phrased, or misunderstood the question. No other metric in this system can detect that.

How it is tracked: After each completed assistant response, a "Helpful?" thumbs-up / thumbs-down widget appears below the message in the UI. Clicking a thumb POSTs `{request_id, rating}` to `/feedback`, which writes a row to the `feedback` Supabase table. The `request_id` is returned by the backend in the `done` SSE event and stored on the frontend message object, linking each rating back to the full request record in `chat_logs` for post-hoc analysis.

The metric is `null` (not zero) when no ratings have been submitted yet, to distinguish "no data" from "universally disliked."

Interpretation: A satisfaction rate above 0.80 indicates the system is handling the question mix well. A drop correlated with a rise in `tool_invocation_rate` would suggest tool-routing errors; a drop uncorrelated with tool use would point to answer quality or phrasing issues.

6.2 Operational metric: tool invocation rate

What it is: The fraction of requests where `tool_calls` is a non-empty array, rounded to four decimal places.

Why it matters: This metric characterizes the question distribution and validates that the model is using tools when appropriate. A value near `0.0` on a dataset that should require lookups signals under-use (the model may be confabulating). A value near `1.0` on a mixed dataset signals over-use (unnecessary latency and cost).

How it is tracked: Every completed request writes the full `tool_calls` array to `chat_logs`. The metric sums rows where `jsonb_array_length(tool_calls) > 0` divided by total rows.

6.3 Supporting context values

Value	Source	Purpose
`total_requests`	`chat_logs` row count	Sample size for the two primary metrics
`avg_latency_ms`	Mean of `duration_ms`	Operational baseline; rises when tool call chains are long
`total_ratings`	`feedback` row count	Denominator for `satisfaction_rate`; shows how much signal has been collected

6.4 Latency interpretation

Tool invocation rate and latency are coupled — requests that invoke tools have higher latency than direct answers. The average therefore mixes two populations:

Segment	Expected latency
Direct answers (no tools)	1–4 s
Single tool call	4–12 s
Chained tool calls	8–20 s

This decomposition is derivable from `chat_logs` using `jsonb_array_length(tool_calls)` to bucket by tool call count, without schema changes.

6.5 Future metrics

- **Tool error rate** — `COUNT(*) FROM tool_error_logs` / total tool invocations. Tracks reliability of each tool independently.
- **Per-tool satisfaction breakdown** — join `feedback` → `chat_logs.tool_calls` to see whether ratings differ for tool-using vs. direct-answer responses.
- **p95/p99 latency** — Postgres `percentile_cont(0.95) WITHIN GROUP (ORDER BY duration_ms)` exposes tail latency without application changes.

---

7. Evaluation

7.1 Functional correctness

Manual testing covered four question categories:

Category	Example	Expected behavior
Direct knowledge	"Who wrote Hamlet?"	Model answers without calling any tool
Current facts	"What is the price of Bitcoin today?"	`web_search` invoked; answer cites the result
Arithmetic	"What is sqrt(2) * pi to 6 decimal places?"	`calculator` invoked; exact result returned
Chained	"What is the current gold price? Square it."	`web_search` then `calculator` invoked in sequence

The chained case is the most important to validate: it requires the model to recognize that the output of one tool is the input to another, and to structure the tool calls accordingly without explicit instruction.

7.2 Tool selection accuracy

A correct tool selection is one where:

- The model calls a tool when external data or exact computation is needed.
- The model does *not* call a tool when the question is answerable from training knowledge.

False positives (unnecessary tool calls) increase latency and cost without improving answer quality. False negatives (missing tool calls) produce answers with hallucinated facts or imprecise arithmetic. The current implementation does not measure tool selection accuracy automatically; it requires labeling a test set of questions with ground-truth tool selections.

7.3 Tool failure handling

The `is_error` flag on `tool_result` messages tells the model that a tool failed. The expected behavior is that the model acknowledges the failure in its response rather than fabricating a result. To test this, the calculator tool can be triggered with an invalid expression:

```
User: "What is log(of the moon)?"
Expected: calculator called with an invalid expression → tool_error logged →
          model responds explaining the calculation is not possible
```

This was verified manually; the model correctly surfaced the tool failure in its response rather than inventing an answer.

7.4 Satisfaction ratings as evaluation signal

The thumbs-up / thumbs-down widget attached to every assistant response provides a lightweight, continuous evaluation layer on top of the manual scenarios above. Rather than a one-time benchmark run, it accumulates signal over normal use.

Representative early observations:

- Direct-knowledge questions (e.g., historical facts) consistently receive thumbs-up — the model's training data is sufficient and the answer is immediate.
- Chained questions (web search → calculator) receive thumbs-up when the sourced value is current and the arithmetic is correct, and thumbs-down when the web search returns a stale or ambiguous price.
- Calculator failures where the model explains it cannot evaluate the expression receive mixed ratings — some users find the explanation helpful, others expect a numeric answer regardless.

These patterns illustrate that satisfaction rate is more sensitive to answer content than tool selection. A low rating on a tool-using response most often reflects data quality (stale search result) rather than a wrong routing decision.

7.5 Limitations of the current evaluation

Manual testing covers the happy-path scenarios. There is no automated test suite, no golden dataset, and no regression detection. A production system would require:

- A labeled evaluation set covering all four question categories with ground-truth answers.
- An automated harness that runs the eval set against the API and compares responses.
- Metrics tracked over model version changes to detect regressions.

The Supabase `chat_logs` table already stores the inputs, tool calls, and responses needed to build such a dataset retroactively from production traffic. The `feedback` table's `request_id` foreign key means any future labeling effort can be joined to the full interaction record.

---

8. Deployment

8.1 Backend on Render

The backend is deployed as a Render Web Service with the repository root set to `backend/`. The `Procfile` specifies the start command:

```
web: uvicorn main:app --host 0.0.0.0 --port $PORT
```

Render injects `$PORT` at runtime. `--host 0.0.0.0` is required so the process binds to all network interfaces, not just loopback. The service uses Render's free tier, which spins down after 15 minutes of inactivity — cold starts on this tier add 30–60 seconds to the first request after a period of no traffic.

Environment variables (`ANTHROPIC_API_KEY`, `SUPABASE_URL`, `SUPABASE_KEY`) are set in the Render dashboard and are never committed to the repository.

8.2 Frontend on Vercel

The frontend is deployed on Vercel with the repository root set to `frontend/`. Vercel auto-detects Next.js and runs `next build` with no additional configuration. `NEXT_PUBLIC_API_URL` is set to the Render service URL in the Vercel environment variable settings.

`NEXT_PUBLIC_` prefix is required for Next.js to include the variable in the browser bundle. Variables without this prefix are server-only and would be undefined in `Chat.tsx`, which is a client component.

8.3 CORS configuration

Because the frontend and backend are deployed on different domains, the FastAPI backend must explicitly allow the Vercel domain in its CORS middleware:

```
allow_origins=[
    "http://localhost:3000",
    "http://127.0.0.1:3000",
    "https://project-l5gn6.vercel.app",
]
```

The stable Vercel URL (`project-l5gn6.vercel.app`) is used rather than deployment-specific preview URLs, which change on each push. Preview deployments will fail CORS checks if their URL is not added — this is acceptable for this project but would require a wildcard origin or Vercel's deployment protection features in a production setting.

8.4 Secrets management

No secrets are committed to the repository. Backend secrets are in Render's environment variable store; frontend variables are in Vercel's environment variable store. `.env` files are listed in `.gitignore`. `.env.example` files in both packages document the required variables and their sources without containing real values.

---

9. Reflection

9.1 What worked well

The agentic loop is simple and robust. The `while True` / `break on end_turn` pattern in `agent.py` handles all three stop reasons cleanly and is easy to extend with new tools. Adding a new client-side tool requires only a new branch in `_execute_tool` and a new entry in the `TOOLS` list.

Immediate tool error logging provides useful signal. Separating `tool_error_logs` from the end-of-request `chat_logs` means that a failed tool call is recorded even if the overall request succeeds. In practice, the model recovers gracefully from calculator errors (malformed expressions) most of the time, which would make the errors invisible if they were only recorded in the `tool_calls` JSONB array.

SSE streaming makes the latency feel lower. Multi-tool requests can take 10–15 seconds end to end. The streaming architecture ensures text appears incrementally and tool call cards update in real time, so users see activity rather than a blank screen.

`request_id` correlation is the right design. Generating the UUID in `main.py` and threading it through to both logging tables makes cross-table debugging straightforward. This pattern scales to additional tables (e.g., per-token streaming logs) without schema changes.

9.2 What would be done differently

Use an async Supabase client. Wrapping synchronous Supabase calls in `asyncio.to_thread` works but adds thread pool overhead and complexity. The official `supabase-py` library includes an async client; using it

directly would simplify the logging code and remove the thread pool dependency.

Add request-level error logging to `chat_logs`. Currently, if the loop raises an unhandled exception the `error` column in `chat_logs` records the exception message but not the stack trace. A structured error object with `type`, `message`, and `traceback` would make post-mortem debugging faster.

Pagination on `/metrics`. The current implementation fetches all rows from `chat_logs` and aggregates in Python. This works at small scale but will degrade as the table grows. The correct fix is a Postgres aggregate query using `AVG(duration_ms)` and a conditional count, which runs in the database and returns two numbers regardless of table size.

Handle `NEXT_PUBLIC_API_URL` more robustly. The frontend falls back to `http://localhost:8000` if `NEXT_PUBLIC_API_URL` is not set. A missing environment variable in a production Vercel deployment would silently send requests to a localhost address that doesn't exist, producing a confusing `fetch` error. A runtime check that throws a clear error message at startup would be more debuggable.

9.3 Potential extensions

Authentication. The current application has no user accounts. Adding authentication (e.g., Supabase Auth) would enable per-user conversation history, personalised system prompts, and usage quotas.

Conversation persistence. The frontend currently sends the full conversation history as a request parameter on every turn. For long conversations this grows without bound. Storing conversation history in Supabase and retrieving it by session ID would decouple message history from the request payload.

Additional tools. The tool registry (`TOOLS` list + `_execute_tool` dispatch) is designed for extension. Useful additions would include a code execution tool, a unit conversion tool, and a database query tool for structured data retrieval.

Streaming tool results. The current design streams text deltas but not tool result content. For web search results, which can be long, streaming the result into the UI before the model begins its synthesis would further reduce the perceived latency of multi-tool responses.

Prompt caching. For applications with a long, stable system prompt, Anthropic's prompt caching feature can reduce input token costs by up to 90% on repeated requests. The `TOOLS` list and any fixed system prompt are good candidates for cache breakpoints.